

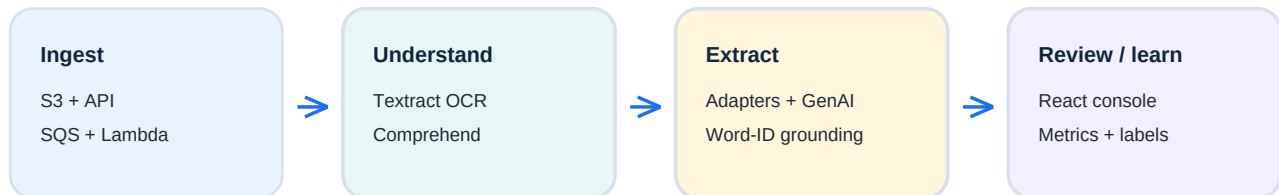
Intelligent Document Understanding

IW Computer Vision Services

A project briefing and pitch guide

The one-line story

We built an AWS-native platform that turns messy benefits documents into structured, reviewable data using OCR, classification, carrier-aware extraction, ML, GenAI, and human feedback.



Prepared for project understanding, technical discussions, interviews, and stakeholder pitches.

Repository resolved in this session: C:\Users\msheriff\Businessolver_data_science_project\iw-computer-vision-services

01

Executive summary

What the project is, in plain language

This project is an intelligent document understanding service for benefits administration. Its job is to take an uploaded image or PDF, understand what kind of document it is, find the relevant information, extract it into a consistent schema, and make the result available for downstream systems and human review.

This is not only an OCR wrapper and not only a single model. It is a platform for operating document intelligence: ingestion, asynchronous processing, model training, strategy authoring, evaluation, annotation, metrics, persistence, security, and deployment are all part of the product.

The strongest pitch: the platform converts document variability into a manageable, measurable, and continuously improvable ML workflow.

Question	Answer
What are we solving?	Manual and brittle processing of heterogeneous insurance and benefits documents.
What did we build?	A production-oriented AWS platform plus a React Computer Vision Console for training, testing, annotation, review, and administration.
How does it work?	S3/API intake -> SQS/Lambda processing -> Textract OCR and layout -> Comprehend classification -> extraction strategy -> DynamoDB/S3 results -> human review and metrics.
What is the technical idea?	Use the right tool at each stage: deterministic cloud services and ML for structure, GenAI for semantic mapping, and exact Textract word IDs for grounding.
What do we gain?	A reusable platform, faster iteration, measurable quality, safer review workflows, and a path from R&D; experiments to cloud-native production.

02

The problem we are solving

Why benefits documents need more than OCR

Benefits administration receives documents from different carriers, employers, providers, and systems. The same business concept may appear in different layouts, labels, tables, or page structures. Important information is often embedded in semi-structured tables rather than clean digital fields.

Challenge	Why it is difficult	What the platform must do
Carrier variation	Aetna, Cigna, Anthem, and other sources use different headers, column positions, and wording.	Resolve a document type and apply a carrier/template-aware strategy.
OCR and layout noise	Rows can merge, cells can split, and text can be spatially meaningful.	Preserve Textract blocks, word IDs, table structure, and geometry.
Multiple tables	A document may contain summaries, payments, contact information, and one relevant service ledger.	Identify the correct table before extracting fields.
Semantic ambiguity	A date may be merged into a description; several amount columns may look plausible.	Use strategy rules, examples, and/or GenAI to map meaning.
Quality and trust	A plausible value that cannot be traced to the source is hard to audit.	Ground predictions in source words and retain evidence for review.

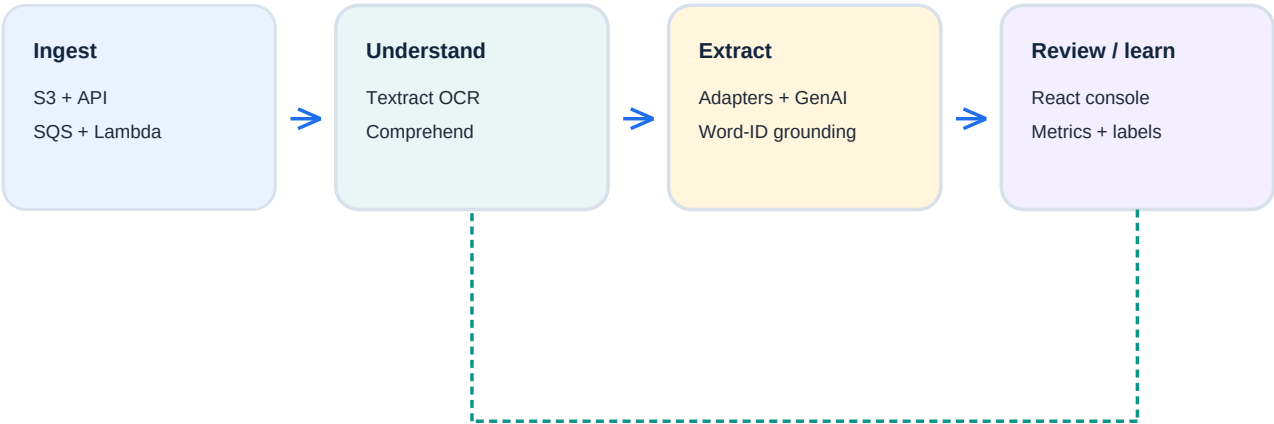
The business consequence is straightforward: without automation, humans spend time locating and transcribing information; with brittle rules, every new layout becomes engineering work. The target is not blind automation. The target is high-confidence automation with a controlled path to human review when the system is uncertain.

The project is solving variability: many document forms, many layouts, one operational workflow.

03

What we built

The platform capabilities



Human labels and evaluation feed the next strategy revision

Capability	What it does	Why it matters
Document ingestion	Accepts uploaded documents through pre-signed S3 URLs and creates processing jobs.	Separates upload from processing and supports asynchronous work.
Classification	Runs Textract text extraction and an Amazon Comprehend custom classifier to assign document types.	Routes each page/document to the correct extraction logic.
Extraction strategies	Uses a factory/resolver pattern for carrier and document-specific strategies; supports Textract adapters and GenAI components.	Keeps document differences behind explicit, testable contracts.
Model lifecycle	Provides training start, evaluation, model metadata, model versions, and strategy revisions.	Turns experimentation into repeatable operations.
Computer Vision Console	React UI for document review, annotation, template administration, model creation, strategy authoring, and metric dashboards.	Gives data and operations users a working loop instead of code changes for every review.
Persistence and callbacks	Stores jobs, documents, annotations, outputs, model records, and test results across DynamoDB and S3; sends configured callbacks.	Creates an auditable system boundary for downstream consumers.

04

How the system is built

Cloud-native architecture and responsibilities

The repository is organized into separate modules so application logic, infrastructure, experimentation, and user experience evolve together without becoming one undifferentiated codebase.

Layer	Repository area	Role
Runtime services	lambda/src	Python APIs, inference workers, domain models, repositories, AWS abstractions, extraction strategies, and model integrations.
Infrastructure	cdk/app	AWS CDK constructs for API Gateway, Lambda, ECS/Fargate, Step Functions, SQS, S3, DynamoDB, IAM, Secrets Manager, CloudFront, WAF, and logging.
Console	webapp/src	React/TypeScript browser application for review, annotation, model operations, strategy authoring, and metrics.
R&D; and evaluation	notebooks and lambda/test/integration	Data analysis, clustering, synthetic data, carrier experiments, GenAI harnesses, and integration tests.
API and design contracts	docs/api and docs/design	API examples, security flow documentation, and design specifications for composite extraction and lifecycle controls.

AWS services are used for their natural strengths: S3 for documents and objects, SQS for buffering and retries, Lambda for event-driven endpoints and workers, ECS Fargate for longer training/evaluation tasks, Step Functions for orchestration, DynamoDB for operational state, Secrets Manager for credentials, and CDK for repeatable infrastructure.

The browser-facing console uses a server-side session flow. The documented security design keeps long-lived AWS credentials and CAS tokens out of browser JavaScript and uses same-origin API routing with an authorizer and server-side session/token state.

05

End-to-end processing flow

What happens to one document

At inference time, the platform follows a clear sequence. The exact strategy can vary by document type, but the control flow remains consistent.

Step	Processing action	Output
1. Upload	Client receives a pre-signed S3 URL and uploads the source image/PDF.	Staged document in S3.
2. Queue	API Lambda creates a processing job in DynamoDB and sends an SQS message.	Durable, retryable job.
3. Normalize	Worker validates MIME type/page limits, converts pages as needed, and creates the internal document model.	Page-level classified-document records.
4. OCR	Textract provides text, tables, forms, geometry, and provider-specific responses as required by the strategy.	Structured OCR and layout context.
5. Classify	Comprehend scores the document/page against trained document classes, unless classification was supplied.	Highest-scoring document type.
6. Resolve strategy	A factory/resolver selects the appropriate adapter, custom model, composite strategy, or GenAI profile.	Executable extraction plan.
7. Extract	Components produce form fields and field groups. Outputs are scoped to owned targets and normalized.	Structured model-labeled data.
8. Persist	Outputs, page data, document metadata, and test results are written to DynamoDB/S3; jobs complete and callbacks are sent.	Downstream-ready result plus reviewable evidence.

The worker is designed for operational reality: it tracks job status, retries transient failures, handles known document failures explicitly, requeues throttled Comprehend work, cleans temporary files, and reports terminal failures to downstream systems.

The intelligence layer

Classification, extraction, and model selection

The platform uses a layered approach because no single technique is ideal for every part of the problem.

Layer	Technique	Best fit
Text and layout acquisition	Amazon Textract OCR, tables, forms, queries, and adapters.	Turning images/PDFs into words, cells, relationships, and geometry.
Document classification	Amazon Comprehend custom classifier.	Identifying document classes so the correct template and extraction strategy can be selected.
Template/carrier extraction	Python strategy classes and Textract custom-query adapters.	Stable fields and known document layouts where deterministic logic is valuable.
Table selection	Heuristics, header matching, and custom table models including the documented XGBoost approach.	Choosing the relevant table in multi-table dental or benefits statements.
Semantic extraction	GenAI model components with carrier-specific profiles and few-shot messages.	Mapping full table context to row-level fields when rules are brittle.
Validation	Target ownership, schema checks, known-word-ID validation, confidence thresholds, and metric materialization.	Preventing silent wrong outputs and making quality measurable.

Explain the design this way: deterministic systems handle repeatable structure; learned models handle patterns; GenAI handles semantic ambiguity; validation and human review handle uncertainty.

Older pattern	Platform direction
One carrier-specific rule path per layout	Reusable strategy catalog/profile with explicit target fields and model components.
LLM returns text that may be plausible but ungrounded	LLM returns Textract word IDs that can be verified and rehydrated to text and bounding boxes.
Manual review disconnected from training	Console annotations, dataset selection, training, evaluation, and review are one operating loop.

07

The key GenAI innovation: word-ID grounding

How semantic extraction becomes auditable

The GenAI extraction approach does not ask the model to invent final text values. It gives the model the Textract table structure and a word dictionary, then requires the model to return lists of existing Textract word IDs for each field.

Input to model	Model response	Server-side handling
Full table rows and cells	claim_details array	Parse JSON and enforce the expected schema.
Each cell's text and word_ids	Field values as word-ID lists	Reject non-list values and unknown IDs.
Carrier-specific field rules and examples	Description/date/amount mappings	Rehydrate IDs into original Textract WORD blocks.
Known word-ID set	No guessed source identifiers	Produce text, confidence, geometry, and reviewable annotations.

Example contract

```
{"claim_details": [{"description": ["w-20", "w-21", "w-22"], "start_date": ["w-24"], "amount": ["w-27"]}]}
```

If the model returns an ID that does not exist in the supplied Textract context, the extraction path raises a validation error rather than quietly accepting a hallucinated value. The IDs are rehydrated back to source blocks and geometry for review.

Pitch this as grounded extraction: GenAI decides which source tokens answer the question; the platform verifies that every token came from the document.

Benefits

- Reduces fabricated values because outputs must point to known source words.
- Bounding boxes and locations come from Textract instead of being guessed by the model.
- Supports text and spatial-overlap evaluation against human-labeled spans.
- Lets a reviewer see exactly which words produced each field and correct the result.

08

Composite extraction and strategy authoring

How multiple model components work together

The newer architecture treats extraction as a composition of model components rather than one monolithic model. A strategy owns an exact set of model revisions, their output targets, and dependency relationships.

Lifecycle	Meaning
DRAFT	Strategy definition can be authored, trained, and evaluated, but cannot be referenced by a live template.
TRAIN	Trainable components use the strategy's train cohort; inference-only components can be marked NOT_REQUIRED.
EVALUATE	Components and the final composite run on the held-out test cohort and write metrics/results.
SEALED	Explicit service-side transition after required work completes; experiment-defining fields become immutable.
PROMOTE	Only a sealed strategy revision can be attached to a template or used as the active extraction path.

Concept	Meaning
ExtractionStrategy	Experiment and execution definition for a document type: cohort, composition, status, evaluation metrics, and version.
ExtractionModel	Immutable model revision owned by exactly one strategy revision; may be TEXTRACT_ADAPTER or GENAI.
Output targets	Form fields or field groups that a component owns; collisions are rejected.
Dependency graph	Components can depend on other components; exact revisions and cycles are validated.
Web console	Supports selecting a document type, creating a strategy, creating/adopting models, mapping outputs, saving revisions, starting tests, and inspecting task status.

Human-in-the-loop labeling and evaluation

Why the platform can improve over time

The Computer Vision Console is not only a dashboard; it is an operating surface for the model lifecycle. It connects review and annotation to dataset selection, training, testing, metrics, and controlled promotion.

Human/operator action	Platform response	Learning value
Review classification	Shows document/page and classification outputs; allows correction and review.	Improves class labels and routing quality.
Review extraction	Shows fields/field groups, text, source blocks, spatial evidence, and JSON/page output.	Creates high-quality ground truth for extraction.
Define templates	Manages document types, fields, groups, validation, normalization, and targets.	Makes the output schema explicit and reusable.
Select cohorts	Separates train, validation, and test documents and records dataset metadata.	Supports repeatable evaluation and reduces leakage risk.
Run evaluation	Persists per-document test results and aggregate metrics; exposes dashboards and failure views.	Shows where the model fails instead of hiding averages.
Promote a revision	Requires completed training/evaluation and explicit status transitions.	Creates operational safety around production changes.

The webapp includes confusion matrices, threshold graphs, radar metrics, extraction quality histograms, field failure matrices, cohort inspection, document review, and strategy authoring. That breadth supports both model selection and root-cause analysis.

Humans are not merely a fallback because the model failed; they are part of the data and quality-control system that makes the model useful.

Documented results and impact

What the project has gained

The repository's portfolio document records the following project outcomes. These are documented project metrics; the raw evaluation artifacts are not all present in this repository snapshot, so verify the exact denominator, cohort, and metric definition before using them externally.

Area	Documented result	How to explain it
Multi-carrier medical EOB extraction	91%+ overall; Aetna 91.7%, Cigna 93%+, Anthem 90%+.	Grounded GenAI table extraction improved robustness across different carrier layouts.
Dental table selection	F1 / precision / recall documented as 1.0 on 200 training documents, 653 tables, and 4 categories.	A targeted ML selector narrows 5-10 tables to the relevant service ledger before GenAI extraction.
Baseline comparison	Portfolio reports approximate gains from about 65-70% rule-based accuracy to about 90%+ for the newer approach.	The gain came from full-table context, carrier profiles, and grounded output.
Manual review	Portfolio reports a reduction from 100% review to under 10% for the documented flow.	Automation handles high-confidence cases while uncertain cases remain reviewable.
Iteration speed	Portfolio reports new carriers can be added with configuration/profile changes in hours rather than weeks of code.	Strategy catalog/profile design moves layout differences into versioned configuration.
Latency and cost	Portfolio reports under 5 seconds/document and about \$0.03/document for the GPT API flow.	Pre-filtering reduces unnecessary model calls; validate these figures for the deployment environment.

What is technically gained

- A repeatable path from upload to structured data and downstream callbacks.
- A reusable strategy/model registry with explicit revisions, ownership, targets, and promotion gates.
- A human-labeling loop that turns corrections into training and evaluation data.
- A grounded GenAI pattern that makes output traceable to source words and locations.
- Infrastructure-as-code and testing foundations for deployment across environments.

11

What you personally gained

Skills and experience this project demonstrates

Skill area	Evidence	Pitchable takeaway
Applied ML	Classification, table selection, custom extraction, threshold calibration, and metrics.	I connect model choice to data shape, operating constraints, and measurable quality.
GenAI engineering	Prompt contracts, profiles, few-shot messages, JSON-only output, word-ID grounding, and validation.	I use GenAI where semantic flexibility matters, but wrap it in deterministic contracts.
System design	Event-driven ingestion, queues, retries, orchestration, workers, persistence, callbacks, and review.	I design around the full operational lifecycle, not just the model call.
Cloud engineering	Lambda, S3, SQS, DynamoDB, ECS/Fargate, Step Functions, Secrets Manager, CloudFront/WAF, and CDK.	I can turn an ML proof into a deployable, secure, observable service.
Data quality	Human annotation, spatial evidence, train/test cohorts, metrics materialization, and failure inspection.	I treat labels and evaluation design as first-class engineering concerns.
Software engineering	Pydantic models, repositories, factories, Docker runtimes, unit tests, integration harnesses, and React UI.	I build maintainable interfaces between models, services, infrastructure, and users.
Product thinking	Admin console, explicit review, model promotion, and documented trade-offs.	I optimize for adoption and trust, not only benchmark scores.

The differentiator is end-to-end ownership: you can explain the data problem, model strategy, production architecture, evaluation, and user workflow as one connected system.

Pitch-ready explanations

Use the version that fits the audience

30-second version

I worked on an intelligent document understanding platform for benefits administration. We take images and PDFs such as medical EOBs and dental statements, classify them, extract structured fields from messy layouts, and return reviewable results. The platform combines AWS Textract for OCR and tables, Comprehend for classification, carrier-aware extraction strategies, and GenAI for difficult semantic mappings. A key design choice was grounded extraction: the model returns source word IDs, so every value can be traced back to the original document and checked by the service.

90-second version

The problem was that benefits documents are highly variable: carriers use different layouts, headers, tables, and conventions, while OCR can merge or split content. A rule-only system becomes expensive to maintain and manual review does not scale. We built an AWS-native IDU platform with S3, SQS, Lambda, Textract, Comprehend, DynamoDB, Step Functions, ECS/Fargate, and a React review console. A document is uploaded, queued, OCR'd, classified, routed through the right strategy, normalized, persisted, and exposed for review. For hard table semantics, ML or heuristics identify the relevant table and GenAI maps fields using full-table context. Instead of free text, the model returns Textract word IDs and the service validates them before rehydrating text and geometry.

Three-minute technical version

Start with the business problem, then walk left to right through the architecture. Explain that the platform has two loops: the inference loop and the learning loop. Inference starts with S3/API intake and SQS buffering, uses Lambda to normalize documents, calls Textract and Comprehend, resolves a carrier/template strategy, runs adapter/custom/GenAI components, validates output ownership and word IDs, and writes results to DynamoDB/S3. The learning loop lives in the Computer Vision Console: operators annotate and review documents, define templates, select train/validation/test cohorts, start training or evaluation, inspect metrics and failures, and explicitly promote a sealed strategy revision. The key trade-off is that GenAI provides semantic flexibility but must not be trusted as an unbounded text generator. By returning source word IDs, the system gets grounding, geometry, and measurable overlap evaluation. The result is a platform that treats document variability as configuration and data rather than repeated one-off code.

13

Likely questions and good answers

Prepare for technical follow-ups

Question	Answer
Why not use only OCR?	OCR gives text and layout primitives, but it does not know which document type, table, or field semantics matter. We add classification, routing, and extraction.
Why use GenAI?	It helps with messy semantics such as merged description/date cells and ambiguous amount columns. It is constrained by schemas, profiles, examples, and source-ID validation.
Why not use GenAI for everything?	Table selection and stable fields can be cheaper and more deterministic with heuristics, XGBoost, or Textract adapters. The hybrid design uses GenAI where it adds the most value.
How do you prevent hallucination?	The contract requires known Textract word IDs, the service rejects unknown IDs, and the IDs are rehydrated to source blocks and geometry.
How do you add a new carrier?	Create or update a versioned strategy/profile with target fields, matching behavior, prompt preamble, and examples; then run evaluation and review the result.
How do you know if a model is good?	Use held-out cohorts and metrics at classification, field, document, and spatial levels; inspect confusion matrices, thresholds, failures, and document evidence.
What happens when the model is uncertain?	Confidence thresholds and validation can suppress or reject output; the document remains available for human review instead of silently producing a plausible value.
What was the hardest part?	Aligning schemas, noisy layout, model ownership, evaluation cohorts, user review, retries, persistence, and safe promotion into one platform.

Trade-offs to mention

- Word IDs improve traceability and reduce hallucination risk, but require a rehydration and validation layer.
- Few-shot and configuration-driven profiles iterate faster than fine-tuning for frequent layout changes, but still require disciplined evaluation.
- Asynchronous queues improve resilience and throughput, but introduce job state, retries, and observability complexity.
- Immutable strategy/model revisions improve reproducibility and promotion safety, but require careful ownership and lifecycle rules.

Evidence map and presentation guardrails

What the repository supports directly

Use this page to keep the pitch accurate. The project has a wide surface area, and some files are production runtime code while others are operator-only experiments or design specifications.

Claim type	Repository evidence	How to phrase it
Platform architecture	Root README, module READMEs, CDK constructs, security review flow, and runtime code.	Built an AWS-native IDU platform with asynchronous processing and a review console.
End-to-end inference	<code>analyze_classified_document.py</code> , <code>service/repository/domain</code> packages, and <code>extraction resolver/factory</code> .	The runtime performs normalization, OCR, classification, extraction, persistence, cleanup, retry, and callbacks.
Composite extraction	<code>composite_extraction_strategy.py</code> , <code>extraction strategy/model</code> models, CDK, and UI strategy workspace.	The newer architecture composes Textract adapter and GenAI components with versioned ownership and validation.
Grounded GenAI	<code>genai_table_executor.py</code> , V2 process documentation, integration harnesses, and tests.	The GenAI contract returns known Textract word IDs that are validated and rehydrated.
Human-in-the-loop	<code>webapp document scene</code> , metric dashboards, strategy workspace, APIs, and security flow.	The console supports review, annotation, model operations, and evaluation.
Quantitative outcomes	<code>GENAI_EXTRACTION_STRATEGY_PORTFOLIO.md</code> and experiment documentation.	Project documentation reports 90%+ carrier extraction and other operational metrics; verify raw cohorts before external publication.

Recommended wording for uncertain metrics

Use: In our documented evaluation runs, the newer grounded GenAI flow reached approximately 90%+ extraction accuracy across the cited carrier scenarios. Avoid: presenting one number as a universal production guarantee without naming the carrier, cohort, fields, metric definition, and evaluation date.

Primary files reviewed

- README.md, `lambda/README.md`, `cdk/README.md`, `webapp/README.md`, and `notebooks/README.md`.
- `lambda/src/api/v1/analyze/job/analyze_classified_document.py`.
- `lambda/src/extraction/composite_extraction_strategy.py` and `genai_table_executor.py`.
- `docs/design/security-review/cvc-webapp-data-flow.md`.
- `docs/design/specs/GENAI_EXTRACTION_STRATEGY_PORTFOLIO.md`.
- `lambda/test` integration strategy harnesses and `webapp document/model/strategy` components.

Final takeaway: this project is best presented as a production-minded intelligent document platform with a grounded hybrid ML/GenAI extraction strategy and a built-in human learning loop.